



The Language Guide

A practical reference for C++Extended developers

Language extensions, examples, CMake, projects, and roadmap

ENGLISH DOCUMENTATION EDITION

Project documentation snapshot

cxxe compiler - .cppe source files

C++Extended (cxxe)

A practical language and project guide

What this book covers

C++Extended is documented as a C++ superset: existing C++ remains the foundation, while the language adds ergonomics such as named parameters, decorators, reflection, properties, defer, output redirection, and other extensions. This book collects the documented feature set and turns it into a book-style reference with examples and practical project guidance.

Reading the status labels

Features marked Available are documented as implemented. Testing Phase features are implemented but still subject to change. Concept features are future ideas. Signals are explicitly marked Abandoned.

Project repository

<https://github.com/oscar-soirey/cxx-Extended>

The online repository remains the authoritative place for source code and release artifacts. This PDF is a documentation-oriented snapshot and is designed to remain useful when printed.

Contents

1. Introduction	6
1.1 The C++ superset model	6
1.2 A migration-first mindset	6
1.3 What this guide is and is not	6
2. Getting Started	6
2.1 Your first extended source file	6
2.2 A practical compile pipeline	7
2.3 Project hygiene	7
3. Feature Status	7
3.1 How to read the roadmap	8
4. C++ Compatibility	8
4.1 Interoperability mindset	8
4.2 File-by-file adoption	8
4.3 Keeping boundaries clean	8
5. Named Parameters	9
5.1 Basic syntax	9
5.2 Good use cases	9
5.3 Design guidance	9
6. Native Decorators and Reflection	9
6.1 Registering a class	10
6.2 Creating instances by name	10
6.3 Accessing exposed members	10
6.4 Runtime resolution	10
7. Custom Decorators	10
7.1 A minimal decorator	10
7.2 Decorators with parameters	11
7.3 Applying a decorator	11
7.4 Decorator templates	11
8. Lifecycle and API Decorators	11
8.1 Example	11
9. Signals - Abandoned	12
10. defer	12
10.1 File cleanup	12
10.2 Why this matters	12
10.3 Scope discipline	12

11. Inspectable Enums	13
11.1 Basic declaration	13
11.2 Practical uses	13
12. Properties	13
12.1 Declaration	13
12.2 Usage	13
12.3 When properties help	13
13. Output Redirection with IOutput	14
13.1 Documented output families	14
13.2 Output destinations	14
14. CMake and the Toolchain	14
14.1 Recommended project layout	14
14.2 Toolchain integration pattern	14
14.3 Why the toolchain approach works	15
14.4 Out-of-tree generation	15
15. Using C++Extended in a Real Project	15
15.1 Example architecture	15
15.2 Where the extensions fit	16
15.3 A representative component	16
15.4 Migration strategy for teams	16
16. Practical Guides	16
16.1 Guide: Designing a reflective type	16
16.2 Guide: Adding a safe property	16
16.3 Guide: Resource lifetime with defer	17
16.4 Guide: API evolution	17
16.5 Guide: Output abstraction	17
17. Dynamic Values and Pattern Matching	17
17.1 The dynamic type	17
17.2 Dynamic values in containers	17
17.3 Pattern matching	18
17.4 Testing these features in a project	18
18. The Future of C++Extended	18
18.1 Testing-phase direction	18
18.2 Conceptual data formats	18
18.3 Automatic serialization	19
18.4 Design principle for future features	19
19. Build, Test, and Documentation Workflow	19

19.1 Reproducible builds	19
19.2 Continuous integration	19
19.3 Documentation as part of the API	19
20. Notes, Limitations, and Compatibility Practices	19
20.1 Version-sensitive APIs	20
20.2 Generated-code troubleshooting	20
20.3 Experimental syntax	20
21. Quick Reference	20
22. Appendix: Project Resources	20

1. Introduction

C++ is powerful, but it carries decades of ceremony: plumbing to observe a variable, boilerplate for getters, fragile macros for converting an enum to a string, and patterns that can make scope cleanup harder to express than the intent itself.

C++Extended (compiler: `cxxe`) is presented as a C++ superset that keeps the original language and adds a layer of modern ergonomics. The central idea is deliberately conservative: extend the language you already know instead of replacing it with a separate ecosystem.

The project documentation emphasizes a migration-friendly model. Since valid C++ remains valid C++Extended, an existing codebase can be adopted incrementally rather than rewritten all at once.

The rest of this guide follows that philosophy: first understand the model, then learn each extension, then use the pieces together in a real build.

1.1 The C++ superset model

The language is intended to sit above ordinary C++. Existing headers, standard library types, classes, functions, templates, and the normal C++ programming model remain relevant. Extended syntax is compiled down toward standard C++ rather than replacing it with an unrelated runtime model.

1.2 A migration-first mindset

- Keep an existing C++ project and migrate individual source files where the extra syntax is useful.
- Use ordinary C++ whenever it is already the clearest solution.
- Use C++Extended features for repetitive patterns: metadata, named arguments, scope cleanup, properties, and reflection.

1.3 What this guide is and is not

This document is a practical companion to the project documentation. It intentionally distinguishes implemented features from experimental and conceptual ones. For exact compiler behavior, syntax details, and the newest release information, consult the repository associated with the project.

2. Getting Started

The smallest useful workflow is straightforward: install or build the C++Extended compiler, keep your normal C++ toolchain available, write a file using the C++Extended syntax you need, and let `cxxe` produce standard C++ for the normal compiler/linker pipeline.

2.1 Your first extended source file

Example: a valid C++ program using a C++Extended feature

```
#include <iostream>

int add(int a, int b);

int main() {
    int result = add(b=5, a=4);
    std::cout << result << std::endl;
    return 0;
}
```

The example deliberately remains ordinary C++ except for named arguments. That is the intended style: use the extension where it improves readability, without giving up familiar C++ syntax elsewhere.

2.2 A practical compile pipeline

Conceptually, a C++Extended build has three layers:

- Extended source: the source files that use C++Extended syntax.
- Translation: cxxe lowers the extensions toward standard C++.
- Native build: your regular C++ compiler and linker finish the executable or library.

2.3 Project hygiene

Keep generated C++ in a build or generated directory rather than mixing it into the hand-written source tree. This makes source control cleaner, keeps compiler output obvious, and makes it easier to switch back and forth between extended and conventional source.

Tip

Treat generated C++ as build output unless the project explicitly chooses to vendor it. Hand-editing generated files generally defeats the purpose of the translation step.

3. Feature Status

The project documentation currently distinguishes implemented, testing, conceptual, and abandoned work. These labels matter because they tell readers how strongly they can depend on a feature in a long-lived project.

Feature	Status
All C++ features	Available
Named Parameters	Available
Native Decorators (@register, @exposed, ...)	Available
Custom decorators	Available
@deprecated, @since, @experimental	Available
Signals (signal:)	Abandoned
defer	Available
Inspectable Enums	Available
Properties (property)	Available

Feature	Status
Output Redirection (IOutput)	Available
CMake Toolchain	Available
Decorator templates	Available
Type dynamic	Testing Phase
Pattern matching (match)	Testing Phase
JSON / XML / other formats	Concept
Automatic serialization (@serializable)	Concept

3.1 How to read the roadmap

Available features form the core documented surface. Testing Phase features should be used with versioning discipline because syntax or behavior may still change. Concept features are design directions rather than promises of a ready-to-use implementation.

Important

Signals appear in older examples because the syntax is part of the historical documentation, but the current feature-status table marks them as Abandoned. They should not be treated as a current language feature.

4. C++ Compatibility

Compatibility is more than a slogan: it is the organizing principle of the language. C++Extended is described as a superset, so ordinary C++ remains the baseline rather than a legacy mode.

4.1 Interoperability mindset

- Use normal C++ classes and standard library containers in extended code.
- Keep familiar headers, namespaces, templates, operators, and control flow.
- Add C++Extended syntax only where it expresses intent more directly.

4.2 File-by-file adoption

A practical migration strategy is to start with one low-risk component: a utility module, a small class hierarchy, or a subsystem where repetitive patterns are already obvious. Keep the first migration small enough that any compiler or integration issue has a narrow blast radius.

4.3 Keeping boundaries clean

At architecture boundaries - libraries, plugins, C APIs, generated bindings, or public headers - prefer conventional C++ interfaces unless an extended feature is explicitly part of the supported interface contract. This reduces coupling between the language layer and downstream tools.

5. Named Parameters

Named parameters let callers associate each argument with the parameter name rather than relying only on position. The documentation describes them as order-independent.

5.1 Basic syntax

```
int add(int a, int b);

int main() {
    int result = add(b=5, a=4);
    return result;
}
```

The call reads as a mapping between names and values. That can be useful when several parameters share the same type or when the semantic meaning of a literal is otherwise easy to miss.

5.2 Good use cases

- Configuration-like functions with multiple same-type arguments.
- API calls where call-site readability is more valuable than positional brevity.
- Code review situations where the parameter meaning should be obvious without jumping to the declaration.

5.3 Design guidance

Named arguments are most useful when parameter names are stable and descriptive. Avoid treating them as a replacement for a well-designed type when a parameter set has become large enough to deserve its own object.

6. Native Decorators and Reflection

Native decorators add metadata and controlled runtime access. Two documented decorators are central to the reflection example: `@register` registers a class with a factory, while `@exposed` marks a member for runtime access by name.

Feature	Status
<code>@register</code>	Registers a class in the factory.
<code>@exposed</code>	Exposes a member to reflection; it must be public.

6.1 Registering a class

```
@register
class A {
public:
    @exposed int health;
};
```

6.2 Creating instances by name

```
// Instantiate by name
A* obj = stde::factory_new("A");
// cxxe transforms this into: new A()

// Find the type by name
A* obj2 = new stde::factory_find("A")();
// cxxe transforms this into: new A()
```

The important idea is not the exact helper spelling but the ability to move from a runtime string to a registered type. The project documentation describes the factory operation as a runtime resolution mechanism.

6.3 Accessing exposed members

```
obj->get_member("health") = 4;
// cxxe transforms this into: obj->health = 4;
```

Rule

An @exposed member must be public.

6.4 Runtime resolution

```
stde::factory_new(unknown_string);
```

The documentation explicitly describes this as a runtime use case: the class name can come from a source that was not known at compile time.

7. Custom Decorators

Custom decorators let developers define reusable wrappers around functions. A decorator receives the decorated function and can run code before and after the call.

7.1 A minimal decorator

```
void @my_decorator() -> func(int a) {
    before();
    func(a);
    after();
}
```

7.2 Decorators with parameters

```
void @my_decorator(/* decorator parameters */) -> func(int a) {
    before();
    func(a);
    after();
}
```

7.3 Applying a decorator

```
@my_decorator
void work(int a) {
    // ...
}
```

7.4 Decorator templates

Decorator templates are implemented. This extends the decorator idea to templated/generic use rather than limiting wrappers to one fixed function signature. The current feature status therefore lists Decorator templates as Available.

Practical pattern

Use custom decorators for cross-cutting behavior: logging, instrumentation, validation, tracing, registration helpers, or other wrappers that would otherwise be repeated around many functions.

8. Lifecycle and API Decorators

Three standard decorators communicate API evolution without hiding the lifecycle in comments alone.

Feature	Status
@deprecated(msg)	Marks an element as deprecated, with a message.
@since(version)	Indicates the version in which an element was introduced.
@experimental(msg)	Marks an unstable API that may change.

8.1 Example

```
@since("1.2")
@deprecated("Use load_v2() instead")
void load();

@experimental("The interface may change")
void new_feature();
```

These annotations are especially useful in libraries where consumers may not read every release note. The metadata stays next to the declaration it describes.

9. Signals - Abandoned

The historical documentation described signal: blocks as a way to detect changes to variables and execute code when a value changed. Examples included globals and class members.

Historical syntax

```
int my_int = 0;

signal: my_int {
    var_changed();
}
```

Historical syntax inside a class

```
class Player {
public:
    int score = 0;

    signal: score {
        update_ui();
    }
};
```

Current status

Signals are explicitly Abandoned in the feature table. Keep these examples for historical understanding only; do not design new code around them.

10. defer

defer executes a block when leaving the current scope, regardless of the exit path. It is therefore a direct way to put cleanup next to the resource acquisition that needs it.

10.1 File cleanup

```
void process() {
    FILE* f = fopen("data.txt", "r");
    defer { fclose(f); }

    // use f here
} // fclose(f) runs here
```

10.2 Why this matters

The value of defer is scope proximity. Instead of putting cleanup far away from acquisition or relying on a destructor solely to simulate a finally-style action, the cleanup instruction stays next to the resource it protects.

10.3 Scope discipline

Use the narrowest practical scope. The closer the acquisition, use, and deferred cleanup are, the easier the lifetime is to verify during review.

11. Inspectable Enums

Inspectable enums add runtime-oriented operations to enumeration values, including enum-to-string and string-to-enum conversion and other reflection utilities.

11.1 Basic declaration

```
enum class Color { Red, Green, Blue };
```

11.2 Practical uses

- Human-readable diagnostics and logging.
- Configuration values loaded from text.
- Editor or tool interfaces where values need names.
- Runtime inspection and generic tooling.

The core documentation lists enum-to-string and string-to-enum conversion as first-class goals; exact helper function names are intentionally not hard-coded here because the project notes that the std:: API may vary between versions.

12. Properties

Properties provide a native getter/setter syntax, allowing an assignment-like API at the call site while keeping validation or transformation logic in the class implementation.

12.1 Declaration

```
class Character {  
    int health;  
  
public:  
    property Health {  
        get { return health; }  
        set { health = clamp(value, 0, 100); }  
    }  
};
```

12.2 Usage

```
Character c;  
c.Health = 150;    // calls the setter: health = 100  
int h = c.Health; // calls the getter
```

12.3 When properties help

Properties are especially useful when a field starts simple but later needs validation, clamping, event hooks, logging, or computed behavior. The public call site does not have to change just because the implementation grows more sophisticated.

13. Output Redirection with IOutput

C++Extended documents native output redirection through an IOutput interface. The intention is to give applications one abstraction through which output can be routed to different destinations.

13.1 Documented output families

- `std::cout`
- `printf`
- `std::print`
- Other standard output mechanisms

13.2 Output destinations

An IOutput implementation can send output to a file, an editor console, a network endpoint, a log, or another application-defined destination. This makes the output policy replaceable without forcing every subsystem to know where its messages will ultimately appear.

Architecture pattern

Treat output as a dependency. A subsystem should write to the configured output abstraction rather than directly deciding whether messages belong in a terminal, a file, or a GUI console.

14. CMake and the Toolchain

C++Extended is documented as having a dedicated CMake toolchain integration. The key workflow is to process extended source before the normal C++ compile and link steps, allowing an existing CMake project to adopt the language without abandoning its established build system.

14.1 Recommended project layout

```
MyProject/  
  CMakeLists.txt  
  src/  
    main.cpp  
    player.cppe  
  include/  
    player.hpp  
  generated/  
  tests/  
  build/
```

The exact extension used by a project can vary with the compiler setup; the important separation is between hand-written source, generated C++, tests, and the CMake build directory.

14.2 Toolchain integration pattern

Illustrative CMake integration pattern - adapt the compiler invocation to the installed cxxe release

```

cmake_minimum_required(VERSION 3.20)
project(MyCxxExtendedProject LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# C++Extended source is translated as part of the build.
# The exact cxxe invocation depends on the installed version.
set(CXXE_COMPILER cxxe)

# Keep generated C++ in the build tree.
# Add the generated files to your normal C++ target.
add_executable(MyApp
    src/main.cpp
    ${CMAKE_CURRENT_BINARY_DIR}/generated/player.cpp
)

# The cxxe generation step should be attached with
# add_custom_command()/add_custom_target() or the project's
# dedicated C++Extended toolchain configuration.

```

14.3 Why the toolchain approach works

- CMake remains responsible for targets, dependencies, include paths, and linking.
- cxxe handles the extended-language translation step.
- The normal C++ compiler remains responsible for platform-specific code generation.

14.4 Out-of-tree generation

Generated files belong in the build or generated tree. This keeps source packages smaller, makes clean builds predictable, and avoids committing compiler artifacts unless a project explicitly requires them.

15. Using C++Extended in a Real Project

A language becomes valuable when it improves a real codebase rather than only demonstrating isolated syntax. This chapter combines the documented extensions into a practical project strategy.

15.1 Example architecture

```

MyApplication/
  CMakeLists.txt
  src/
    main.cpp
    app.cppe
    character.cppe
    reflection.cppe
  include/
    app.hpp
  tests/
    character_tests.cpp
  generated/
  build/

```

15.2 Where the extensions fit

Named parameters improve public API call sites. Decorators can carry registration or lifecycle metadata. Properties provide safe field-like access. `defer` keeps cleanup local. `IOOutput` centralizes logging/output destinations. The result is not a new architecture; it is ordinary C++ architecture with less boilerplate at selected points.

15.3 A representative component

Representative composition of documented features

```
@register
class Character {
    int health = 100;

public:
    @exposed int id;

    property Health {
        get { return health; }
        set { health = clamp(value, 0, 100); }
    }

    void load(FILE* f) {
        defer { fclose(f); }
        // load data
    }
};
```

This example is intentionally architectural rather than a claim about a complete runtime API. It shows how multiple extensions can coexist around a familiar C++ class.

15.4 Migration strategy for teams

- Start with one module and keep generated output isolated.
- Document which features are Available versus Testing Phase in the project contribution guide.
- Keep public APIs stable and use lifecycle decorators when deprecating symbols.
- Add build-system checks so extended source is always regenerated when its inputs change.

16. Practical Guides

16.1 Guide: Designing a reflective type

When a subsystem needs to instantiate types from runtime names, begin with a registered class and explicitly expose only the state that must be visible. The documented `@register` and `@exposed` pair makes the boundary clear.

16.2 Guide: Adding a safe property

Keep the actual storage private, make the property public, and put normalization in the setter. This yields a stable call site even when validation rules evolve.

16.3 Guide: Resource lifetime with defer

Acquire the resource, immediately place the corresponding cleanup in a defer block, and then do the useful work. Avoid delaying the cleanup declaration until after several unrelated operations.

16.4 Guide: API evolution

Introduce new APIs with `@since`, use `@experimental` while an interface is unstable, and add `@deprecated` with a migration message when the replacement is ready.

16.5 Guide: Output abstraction

Define an application-level output destination behind `IOOutput`. Keep application logic independent of whether the destination is a terminal, editor panel, file, network sink, or test capture buffer.

17. Dynamic Values and Pattern Matching

Testing Phase

The project documentation currently describes dynamic and match as implemented features in the testing phase. Syntax and behavior may still change.

17.1 The dynamic type

A dynamic value carries a runtime type. The documented interface includes `is<T>()` for type checks and `as<T>()` for retrieving the stored value as a specific type.

```
dynamic value = 42;

if (value.is<int>()) {
    int x = value.as<int>();
    std::cout << "int = " << x << std::endl;
}

value = 3.14f;
std::cout << value.as<float>() << std::endl;

value = std::string("hello");
std::cout << value.as<std::string>().c_str() << std::endl;
```

17.2 Dynamic values in containers

```
std::vector<dynamic> values = { 42, 3.14f, "hello", true };

for (const auto& value : values) {
    if (value.is<int>())
        std::cout << "int: " << value.as<int>() << std::endl;
    else if (value.is<float>())
        std::cout << "float: " << value.as<float>() << std::endl;
    else if (value.is<const char*>())
        std::cout << "string: " << value.as<const char*>() << std::endl;
    else if (value.is<bool>())
        std::cout << "bool: " << value.as<bool>() << std::endl;
}
```

Runtime safety

The project documentation states that calling `as<T>()` with a type that does not match the stored value results in a runtime error.

17.3 Pattern matching

```
void Show(dynamic value)
{
    match value
    {
        int:
            std::cout << "Integer: " << value << std::endl;

        float:
            std::cout << "Float: " << value << std::endl;

        const char*:
            std::cout << "String: " << value << std::endl;

        default:
            std::cout << "Unknown type" << std::endl;
    }
}

int main()
{
    std::vector<dynamic> values = { 42, 3.14f, "hello" };

    for (const auto& value : values)
        Show(value);

    return 0;
}
```

17.4 Testing these features in a project

Keep testing code explicit about the supported runtime types. Test each documented type branch, test the default branch, and include at least one negative case for `as<T>()` so the runtime-error contract remains visible to maintainers.

18. The Future of C++Extended

The current roadmap combines implemented foundations with experimental work and longer-term concepts. The goal is a coherent extension layer rather than a collection of unrelated syntax additions.

18.1 Testing-phase direction

dynamic and pattern matching are documented as Testing Phase. That means the concepts are implemented enough to exercise in real code, while syntax and behavior remain open to refinement based on testing and feedback.

18.2 Conceptual data formats

JSON, XML, and other data formats are documented as concepts for native language-level handling. These are future-facing ideas rather than current baseline features.

18.3 Automatic serialization

Concept example

```
@serializable
class Config {
public:
    int width;
    int height;
};
```

Automatic serialization would move repetitive serialization/deserialization code into generated behavior. The current documentation labels `@serializable` as a concept, so the example should be understood as a design direction, not a guaranteed available feature.

18.4 Design principle for future features

Future extensions should keep the same test: do they make common C++ intent clearer while preserving the existing C++ model? Features that meet that test can improve ergonomics without forcing developers into an entirely different programming style.

19. Build, Test, and Documentation Workflow

19.1 Reproducible builds

Keep build configuration in version control, keep generated output out of hand-written source directories, and make the extended-source translation step explicit in the build graph. This makes clean builds and CI runs behave like local builds.

19.2 Continuous integration

- Run a normal C++ compilation after the C++Extended translation step.
- Exercise both Available and Testing Phase syntax that the project relies on.
- Keep one small representative sample for each extension so compiler regressions are easy to localize.

19.3 Documentation as part of the API

Lifecycle decorators, feature-status tables, examples, and migration notes are not decoration. They make the stability contract visible to users of the language and to the team maintaining the compiler.

20. Notes, Limitations, and Compatibility Practices

The project documentation notes that exact helper names in the `std` library may vary between versions. This guide therefore focuses on the language-level intent when a library helper is not stable in the source documentation.

20.1 Version-sensitive APIs

When documenting or integrating compiler/runtime helpers, pin the project version and keep the generated artifacts reproducible. Check release notes before relying on helper names that are not part of the core syntax.

20.2 Generated-code troubleshooting

- First determine whether a failure occurred during C++-Extended parsing/translation or in the generated C++ compilation.
- Inspect the generated file when a compiler error points to translated code.
- Keep a minimal input reproducer alongside the failing build when reporting a compiler bug.

20.3 Experimental syntax

For Testing Phase features, isolate assumptions behind small adapters where practical. That makes later syntax changes cheaper to absorb.

21. Quick Reference

Feature	Purpose	Status
Named parameters	Pass arguments by name and in any order.	Available
@register	Register a class in the factory.	Available
@exposed	Expose a public member to reflection.	Available
Custom decorators	Wrap functions with reusable behavior.	Available
Decorator templates	Templated decorator support.	Available
@deprecated / @since / @experimental	Express API lifecycle metadata.	Available
defer	Run cleanup on scope exit.	Available
Properties	Native getter/setter syntax.	Available
Inspectable enums	Enum/string conversion and reflection utilities.	Available
IOutput	Redirect application output.	Available
dynamic	Runtime-typed values with <code>is()</code> and <code>as()</code> .	Testing Phase
match	Runtime type matching for dynamic values.	Testing Phase
signal:	Historical change-trigger blocks.	Abandoned
@serializable	Automatic serialization direction.	Concept
JSON/XML facilities	Native data-format direction.	Concept

22. Appendix: Project Resources

Source Code

GitHub repository - [oscar-soirey/cxx-Extended](https://github.com/oscar-soirey/cxx-Extended)

Releases

GitHub Releases

Documentation notes

This edition is intentionally book-like rather than a copy of the web page. It keeps the documented feature names, statuses, and examples, while adding practical chapters around project organization, CMake workflow, migration, testing, and day-to-day use. Wherever the source documentation leaves an API name unspecified or version-sensitive, this guide says so instead of inventing a stable interface.

End note

C++Extended is most useful when it feels like C++ with fewer repetitive patterns - not like a language that asks you to forget C++.